Parallel Pattern: Map

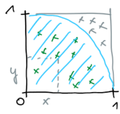
- replicates elemental function over every element of index set
- map replaces iterations of independent loops
- elemental functions should not modify global data that other instances (iterations) depend on
- serial and parallel execution of the map pattern (elements of index set (black) and elemental functions (blue))

  <img src="“figures/map.png” alt="“the map pattern”, width="“50%”">
- examples:
  - gamma correction in images, color space conversions
  - Monte Carlo sampling, ray tracing
- map applies an elemental function to every element of a collection of data in parallel
  - elemental functions should have no side effects
  - no dependency among elements
  - can execute in any order
- embarrassingly parallel
  - one of the most efficient patterns
  - if you have many problems to solve, parallel solution can be as simple as running problems (serial code) on parallel execution nodes
- typically combined with other patterns
  - map does the basic computation, other patters follow
  - reduction, gather = serial random read + map, scan
- scalable implementation of map
  - a lot of care for best performance
  - threads
    - mandatory parallelism
    - separate thread for each element is not a good idea
  - tasks
    - optional parallelism
    - overhead and synchronization at the start and end when elemental functions vary in the amount of work
- the map pattern is a basis for vectorization and parallelization
  - map is related to SIMD, SPMD, SIMT
  - can be expressed as a sequence of vector operations
- parallel for construct in programming languages
  - map is parallelization of the serial iteration pattern where iterations are independent
- if dependencies and side-effects are avoided, map is deterministic

Examples

Monte Carlo π

- Integration
  - random shooting to the square



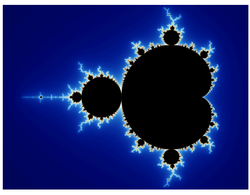
$x^2 + y^2 < 1$

$\pi \approx 4 \cdot \frac{hits}{shots}$

  - statistics of $shots$ and $hits# - $shots$ inside the circular quadrant
  - ration $hits/shots$ is proportional to $\pi/4$
  - slow convergence, relative error is proportional to $1/\sqrt{shots}$
  - a basic unit of work that can be parallelized is one shot
    - a lot of overhead
    - better is to combine several shots to one task
  - solution
    - sources: pimc.c and pisum.c
    - scripts:
      - pimc-1.sh
      - pimc-4.sh and pisum-4.sh

The Mandelbrot set

- an image of The Mandelbrot set from Wikipedia



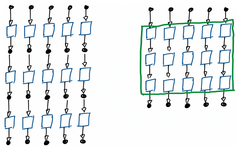
- the Mandelbrot set (black) is the set of all points *c* in the complex plane that do not go to infinity with iterations
  - algorithm

$$z_0 = 0$$
$$z_{k+1} = z_k^2 + c$$
$$\mathrm{count}(c) = \min_{0 \le k < K} (|z_k| \ge 2)$$

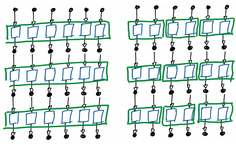
  - Divergence for large z
  - Compute the function up to some maximum value K
- serial control flow in elemental functions
- load imbalance
- implementation
  - opposed to vector operations like ´saxpy',it cannot be efficiently computed on SIMD systems
  - best with SPMD or tiled SIMD

Code Fusion and Cache fusion

- code fusion
  - map of sequence can avoid intermediate memory operations
  - intermediate results are written in registers
  - reduced memory bandwidth, cache and virtual memory problems
  - less synchronization at start/end of execution



- cache fusion
  - maps broken to tiles
  - each tile is executed sequentially on one core
  - tile should fit in cache, avoid accessing main memory
  - loop with predefined size inside a parallel section of a code
  - each map has the same chunk size



Patterns Related to Map

- stencil
  - access to neighbors to get inputs of elemental function
  - important is reading of data
    - data reuse
    - hardware specific for good results (cache size, GPU memory)
- workpile
  - work grows as it is consumed by map
  - could be implemented in OpenMP and OpenCL using explicit work queues
- divide-and-conquer
  - recursive division into smaller parallel subproblems until base is reached, which can be solved sequentially
  - combination of partition and map patterns
  - OpenMP: supported through tasking model
  - GPU programming: poor support for nested parallelism